

Introducing Dr.C: 40+ Years of Csound and Computer Music, Now an Agentic AI Collaborator for Learning and Building Instruments, Effects, Plugins, and Web Apps

Mateo Larrea¹ and Richard Boulanger²

¹ Stanford University, Stanford, CA, USA
`mlarreaf@stanford.edu`

² Berklee College of Music, Boston, MA, USA
`rboulanger@berklee.edu`

Abstract. Dr.C is a domain-specialized AI system for Csound. It exists as two independent products built around the same philosophy. Dr.C CLI is a terminal application, a two-column TUI with chat and version history on one side and a live CSD, waveform, and signal-flow view on the other, built for production: scripting, batch rendering, and shipping advanced audio-reactive web applications that open in a browser. Dr.C Standalone is a desktop application built to teach: it ships with a free-tier model out of the box, keeps sound design and code preview inside one embedded window, and adds three things the CLI does not have. A thumbs up/down feedback loop and a store of durable, user-stated “lessons” quietly steer future generations toward a student’s own taste, without ever modeling how skilled that student is. A force-directed Knowledge Graph page renders 34 people, 15 institutions, 25 techniques, and 12 landmark works of computer music history as an explorable, pannable, zoomable map. A Player page turns a finished `.csd` that exposes its parameters as channels into a MIDI instrument, with MIDI Learn binding a hardware knob to a Csound channel automatically, without manual mapping. Both tools share the same curated retrieval over four decades of Csound community publications, journals, and instrument collections; both propose alternatives rather than finished code; both narrate the history behind a technique as they teach it; and both export working Cabbage plugins. This paper introduces Dr.C as a new kind of collaborator for the Csound community: an AI tutor that draws on the same books, catalogs, and instrument collections its authors have taught from for over forty years, and that is already being integrated into how they teach and how their students explore, design, and compose.

Keywords: Csound, Domain-Specialized AI, Cabbage, Human-in-the-Loop, Educational Scaffolding, Knowledge Graph

1 Introduction

Dr.C is an AI system built specifically for Csound rather than adapted from a general-purpose coding assistant. The case for specialization is straightforward: general models trained on scraped code confuse Csound 6 and Csound 7 syntax, invent opcodes that do not exist, and cannot tell a working `.csd` from one that merely compiles. Dr.C addresses this with curated retrieval, an alternatives-first interaction model that proposes working design options before anything is executed, and validation that catches Cabbage and Csound 7 mistakes a syntax checker alone would miss.

The alternatives-first model and the non-destructive design tree it produces exist to support a navigable design space rather than to correct errors, protecting the freedom to explore the way a composer’s sketchbook or an undo history does (Section 2). Correctness is handled separately, through curated retrieval and the maintained Csound 7 failure-pattern list described below.

Dr.C exists as two products built by the same team but for different rooms. Dr.C CLI (also called Dr.C Terminal) is built toward scripting, automation, and deployable software: audio-reactive web applications, generative pieces, sonification engines. Dr.C Standalone grew in the opposite direction, toward a first-semester student with no programming background who just wants to sit down, describe a sound, and learn something on the way to hearing it. We stopped trying to serve both audiences with one interface, and started asking what each one actually needs.

This paper describes what each tool is, where they diverge in real, shipped features rather than in intent alone, and why we are introducing both to the Csound community now, while they are already changing how we teach and how our students work. We present this as a systems and pedagogy report,

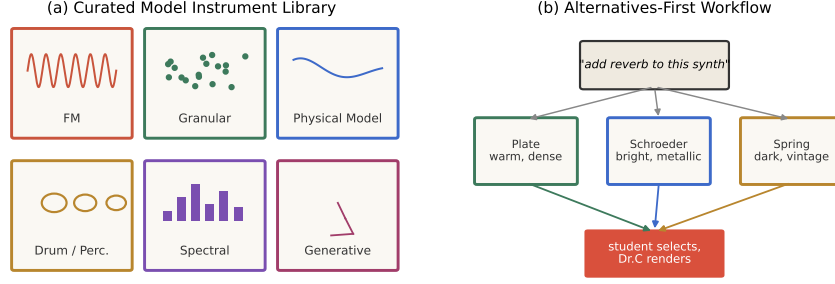


Fig. 1. Shared foundation: (a) the curated model instrument library spanning FM, granular, physical modeling, drum/percussion, spectral, and generative synthesis; (b) the alternatives-first workflow, shown here proposing Plate, Schroeder, and Spring reverb variants for a student’s request, one of which the student selects and Dr.C renders.

grounded in tools already built and classrooms already using them, rather than as the results of a controlled study. Our aim throughout has been to keep the student, not the model, at the center of every design decision, and to treat the interface itself as an open question in Csound pedagogy rather than a settled one.

2 A Shared Philosophy, Two Codebases

Dr.C CLI and Dr.C Standalone are separate implementations, a Bun and TypeScript terminal application versus an Electron and React desktop application, but they are built from the same convictions and much of the same curated material. Both retrieve from the same curated corpus: *The Csound Book* [1], the FLOSS Manual [4], the Cabbage manual and instrument collection [5,6], the McCurdy Csound and Cabbage instrument library [7], the Boulanger Csound Catalog [12], the Csound Journal [9] and Magazine [10], the canonical Csound reference manual [8], and four decades of instruments drawn from the Csound mailing list [13], the CsoundQt example library [11], and historic catalogues by Mathews [14], Risset [15], and Dodge and Jerse [16]. A maintained list of Csound 7 syntax rules and known AI failure patterns helps prevent both tools from proposing deprecated opcodes or constructs carried over from Csound 6. Both also draw on a curated library of working model instruments (Fig. 1a), spanning FM, granular, physical modeling, drum, spectral, and generative synthesis, that a student can open and study directly rather than write from a blank file. Both propose two to four alternatives, typically three, built directly from what a student just asked for rather than a generic starting point, before writing anything to disk (Fig. 1b), and both keep a non-destructive history of every version a session produces, so trying the Schroeder reverb after the plate reverb is never a step backward. Both narrate the history behind a technique as they teach it, mentioning who invented FM synthesis and when granular synthesis first appeared in the same breath as the opcode that implements it today, so a lesson in DSP is also, quietly, a lesson in where the field came from. Both export finished instruments as Cabbage plugins.

Where they part ways is everything downstream of that shared foundation: the retrieval architecture, the feedback mechanism, the model provider, and what happens the moment a student hits render.

3 Dr.C CLI: A Production Tool for Advanced Web Apps

Dr.C CLI [17] treats Csound development the way a programmer treats any other project. Its two-column terminal UI (Fig. 2a) keeps chat and version history on the left and a live CSD view, waveform, and signal-flow diagram on the right. Every session works in an isolated workspace, so nothing touches the real project directory until the student explicitly saves it, and a parameter-locking dialog lets a student freeze specific values while Dr.C works around them. Every alternative Dr.C proposes branches a non-destructive design tree (Fig. 2b) rather than overwriting the working file, so returning to an earlier branch is a restore, not a rewrite. Retrieval is a hybrid of BM25 and vector search

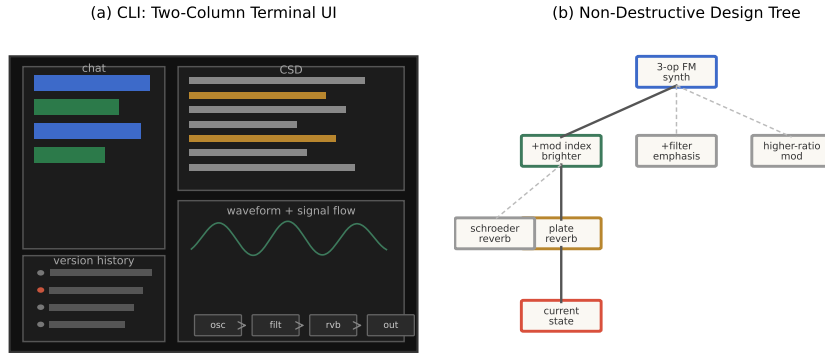


Fig. 2. Dr.C CLI: (a) the two-column terminal UI, chat and version history on the left, live CSD, waveform, and signal-flow on the right; (b) the non-destructive design tree that every proposed alternative branches from, so an earlier version is always a restore away.

over the same Csound Book and example corpus, boosted by ten feedback signals such as compile and render success, audio quality, and which alternative was chosen. Those signals shift what gets retrieved and how it is ranked as a session goes on. Independent sub-agents for synthesis, effects, and modulation run concurrently through a parallel task tool, which is why complex flagship applications, sonification engines, fractal and Mandelbrot explorers, and layered generative pieces can be assembled through parallelized agent workflows rather than entirely by hand. The interface surfaces token use and running cost live as Dr.C works, so a session never runs on an invisible meter. Finished web applications are exported as self-contained HTML using `@csound/browser` and opened in an actual browser tab; there is no embedded preview, because the CLI's web apps are often more elaborate than a single window is meant to hold. The CLI runs against Anthropic, OpenAI, or Google models, or local models through Ollama when a lab lacks reliable connectivity.

4 Dr.C Standalone: A Teaching and Self-Learning Tool

Dr.C Standalone [18] is built to be the CLI's opposite: an education-first, self-teaching environment for a student who wants to learn synthesis by ear, not by file path. There is no terminal, no server, no repository to understand first. A student describes a sound, chooses among Dr.C's alternatives inside the same window, and watches the result play immediately; a generated CSD, web app, or Cabbage-ready VST is auto-detected from the response and rendered as an interactive artifact, one click converts a plain CSD into a web app or a VST without starting over, and the same CSD opens directly in CsoundQt [11] for a student who prefers that IDE. Unlike the CLI, a Standalone web app previews live in an embedded iframe running `@csound/browser`, so a student never leaves the app to hear what they built. Standalone defaults to a free Gemini 2.5 Flash key so a class can start working the day it is installed, with the option to add a paid Anthropic key instead, using Claude Sonnet for full design work and Haiku for fast edits.

Standalone also does three things the CLI does not. Every message carries a thumbs up or down, and that feedback, together with any accepted fix, nudges signed preference weights toward or away from specific techniques and opcodes in future generations; a separate store of durable “lessons”, instructions a student stated once in conversation (“when I ask for a granular texture, keep it tonal”), is extracted automatically and applied to every session after. Deliberately, none of this builds a profile of how skilled a student is; that distinction is left entirely to the student's own choice of Complex, Sine, or Sketch mode, so the system learns a student's taste without ever grading them. Second, a Knowledge Graph page (Fig. 3a) renders a force-directed map, with Louvain community detection coloring related clusters, of 34 people, 15 institutions, 25 techniques, 12 landmark works, and 10 concepts spanning the field. Its point is connection, not inventory: a student can trace Chowning's FM synthesis through the institutions where it was developed to the Csound opcode that implements it today, discovering how the Csound and computer music communities relate to one another rather than reading each name as an isolated entry. Third, the Player (Fig. 3b) turns any finished `.csd` that exposes its parameters as channels into a playable instrument with rotary knobs, a three-octave keyboard, and MIDI Learn: arm a knob, move a hardware control, and Dr.C binds the two together

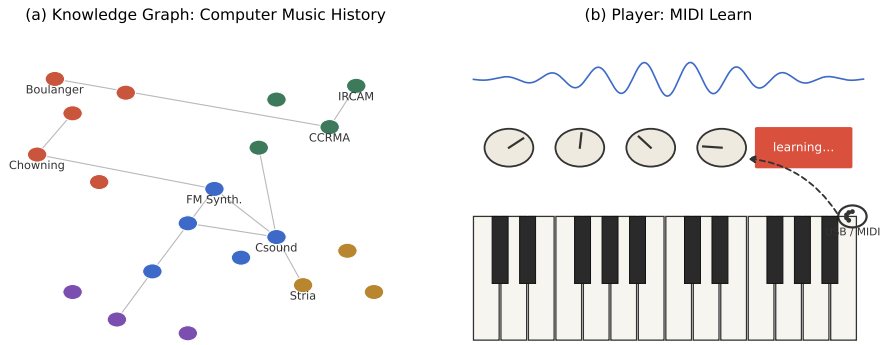


Fig. 3. Dr.C Standalone: (a) the Knowledge Graph, a force-directed map of people, institutions, techniques, and works in computer music history; (b) the Player, which turns any `.csd` into a MIDI instrument and binds hardware controls automatically through MIDI Learn.

automatically, no manual channel mapping. A synthesizer designed minutes earlier becomes something to perform from a MIDI controller.

5 Where They Meet

Both tools converge on one practical endpoint: a loadable Cabbage plugin. Whichever tool designed it, export validates that every Cabbage widget has a matching channel binding and that ranges and layout make sense, so the result can load cleanly as a VST or AU in compatible DAWs. More important than the shared export path is what neither tool does: hand over a finished instrument unasked. Every synthesis and effect decision is something Dr.C proposes and a person accepts, rejects, or asks to see more alternatives for; the interface changes how that conversation happens, never who is doing the composing.

6 Teaching in New Territory

Dr.C is introduced here as a tool the Csound community can adopt and test in its own classrooms and creative practice, to see whether it accelerates learning and design the way it has begun to in ours. The second author, Richard Boulanger, has taught Csound synthesis and composition for over forty years, from *The Csound Book*, personally written tutorials, the Csound Catalog, the CsoundQt example library, and the wider body of instrument-design literature Dr.C now draws on directly; Dr.C is currently being integrated into every project across three of those courses: EP337 (Csound Sound and Plugin Design), EP413 (DSP Spectral Design and Effect Design), and EP461 (Avant-Garde Composition). In these first weeks of integration, our informal classroom observations suggest faster student iteration and creative work, though we make no claim yet about the durability, generality, or effect on learning outcomes. This raises a question this paper does not attempt to answer: does this much assistance deepen a student’s understanding of synthesis, or let them accomplish more while understanding less of what is underneath?

7 Conclusion

Dr.C CLI and Dr.C Standalone share one conviction, that an AI system built for Csound should propose and never dictate, but they answer two different questions. The CLI answers “how do I ship this,” with scripting, version control, and a browser tab. Standalone answers “how do I learn this,” with a feedback loop that learns taste instead of grading skill, a history of the field to wander through, and an instrument to plug a keyboard into before class ends. Both end at a working Cabbage plugin. We are still learning to teach with two doors instead of one. We offer Dr.C to the Csound community in that spirit, as an instrument to test against your own students and your own practice.

Acknowledgements. Dr.C draws on, and gratefully acknowledges, the four decades of Csound community contributions detailed in Section 2. Every text and instrument in that corpus is credited to its author; nothing in Dr.C’s knowledge base is anonymous or unattributed.

References

1. Boulanger, R. (ed.): The Csound Book: Perspectives in Software Synthesis, Sound Design, Signal Processing, and Programming. MIT Press (2000)
2. Lazzarini, V., Yi, S., Heintz, J., Brandtsegg, Ø., McCurdy, I.: Csound: A Sound and Music Computing System. Springer (2016)
3. Lazzarini, V.: Computer Music Instruments: Foundations, Design and Development. Springer (2017)
4. Heintz, J., Hofmann, A., McCurdy, I.: FLOSS Manual — Csound, <https://flossmanual.csound.com/>, accessed June 2026
5. Walsh, R.: Cabbage Audio — Documentation and Examples, <https://cabbageaudio.com/docs/>, accessed June 2026
6. Walsh, R.: Cabbage Instrument Collection and Examples, <https://cabbageaudio.com/>, accessed June 2026
7. McCurdy, I.: Iain McCurdy Csound and Cabbage Instrument Collection, <http://iainmccurdy.org/csound.html>, accessed June 2026
8. The Csound Reference Manual, <https://csound.com/docs/manual/index.html>, accessed June 2026
9. The Csound Journal, <https://csoundjournal.com/>, accessed June 2026
10. The Csound Magazine, <http://www.csounds.com/ezine/>, accessed June 2026
11. Cabrera, A.: CsoundQt Example Library, <https://github.com/CsoundQt/CsoundQt>, accessed June 2026
12. Boulanger, R. (ed.): The Csound Catalog with Audio. csounds.com (2000)
13. Csound Community: Csound Mailing List Archives, <https://csound.com/community.html>, accessed June 2026
14. Mathews, M.V., Miller, J.E., Moore, F.R., Pierce, J.R., Risset, J.C.: The Technology of Computer Music. MIT Press (1969)
15. Risset, J.C.: An Introductory Catalogue of Computer Synthesized Sounds. Bell Telephone Laboratories, Murray Hill, NJ (1969)
16. Dodge, C., Jerse, T.A.: Computer Music: Synthesis, Composition, and Performance, 2nd edn. Schirmer, New York (1997)
17. Larrea, M.: Dr.C CLI (Terminal), <https://github.com/mateolarreaferro/Dr.C>, accessed September 2026
18. Larrea, M.: Dr.C Standalone, <https://github.com/mateolarreaferro/DRC-Standalone>, accessed September 2026